Turning a Recursion into a Loop

The sum $1 + 2 + \cdots + n$, three ways: recursive, with an accumulator, as a loop

Marcus Birkenkrahe

COSC 2230 · CS III · Augustana University

Fall 2026

Why bother?

The recursive `sum` is short, clear, and correct.

Why would you rewrite it as a loop?

Every call costs a stack frame

→ **Switch to editor · code-along 1 · Sum.java**

Counting the pairs in `TwoSum` gave $1 + 2 + \cdots + N = N(N+1)/2$. As a recursion:

```
public static long sum(int n)
{
    if (n == 0) return 0;
    return sum(n - 1) + n;
}
```

n	sum(n)
4	10
10,000	50005000
20,000	StackOverflowError

→ Every call gets its own **stack frame** until it returns. The frames pile up; Java's stack runs out in the tens of thousands.

Read the recursion: three parts and one problem

```
public static long sum(int n)
{
    if (n == 0) return 0;
    return sum(n - 1) + n;
}
```

- **base case:** `n == 0`
- **what it returns:** `0`
- **the recursive call:** `sum(n - 1)`
- **work after the call:** `+ n`

What does `+ n` have to wait for?

→ For `sum(n - 1)` to return. Every frame still owes its `+ n`.

Trace `sum(4)`: down, then back up

Call	n	n == 0?	Returns	Value
`sum(4)`	4	no	`sum(3) + 4`	$6 + 4 = 10$
`sum(3)`	3	no	`sum(2) + 3`	$3 + 3 = 6$
`sum(2)`	2	no	`sum(1) + 2`	$1 + 2 = 3$
`sum(1)`	1	no	`sum(0) + 1`	$0 + 1 = 1$
`sum(0)`	0	yes	`0`	0

→ The calls go **down** the Returns column. The additions happen on the way back **up** the Value column.

A loop can't wait: two moves

A loop has no frames to come back to. So:

Move 1: carry the work along

Rewrite the recursion so that nothing is left to do after the call.

Move 2: rewrite as a loop

Turn that recursion into a `while` loop, in four steps.

A call that is the last thing a method does is a **tail call**. A method whose recursive calls are all tail calls is **tail-recursive**.

→ `sum` is not tail-recursive (`+ n` comes after the call). Binary search is: it skips move 1.

Move 1: carry the work along

→ **Switch to editor · code-along 2 · add sumAcc to Sum.java**

Before: work after the call

```
public static long sum(int n)
{
    if (n == 0) return 0;
    return sum(n - 1) + n;
}
```

After: the call is the last thing

```
public static long sumAcc(int n)
{
    return sumAcc(n, 0);
}
private static long sumAcc(int n, long acc)
{
    if (n == 0) return acc;
    return sumAcc(n - 1, acc + n);
}
```

1. The call passes `acc + n`: the addition moved **inside** the call
2. The base case returns `acc`: when `n` reaches 0, everything has been added
3. The wrapper `sumAcc(n)` only supplies the start value 0 – like `search(key, a)`
4. The helper is `private` ... why?

Why is the helper `private`?

The helper is only correct when `acc` starts at 0.

`sumAcc(4, 7)` returns 17 – a wrong answer, without any error.

`private`: only code in the class can call it. The wrapper is the only way in, and it always starts right.

Trace sumAcc(4, 0): the work happens on the way down

Call	n	acc	n == 0?	Returns
`sumAcc(4, 0)`	4	0	no	`sumAcc(3, 4)`
`sumAcc(3, 4)`	3	4	no	`sumAcc(2, 7)`
`sumAcc(2, 7)`	2	7	no	`sumAcc(1, 9)`
`sumAcc(1, 9)`	1	9	no	`sumAcc(0, 10)`
`sumAcc(0, 10)`	0	10	yes	`10`

→ No Value column: the answer is ready at the base case. Every frame above just passes it on.

When an accumulator is not enough

- It works here because addition doesn't care about order: $0 + 1 + 2 + 3 + 4$ on the way up, $4 + 3 + 2 + 1$ on the way down – both 10
- When the order matters (building a string), the accumulator can turn the result around
- When the pending work is **output** in reverse order, or a method makes **two** recursive calls (mergesort, coming next), there is nothing to add up
- Then the pending work has to be remembered in a **stack** – which is what Java's call stack does for you. That's where stacks come in: our next topic

Step 1 · Which parameters change, and where do they start?

→ Switch to editor · code-along 3 · add sumLoop to Sum.java – don't run before step 4

```
return sumAcc(n - 1, acc + n);
```

Compare the call with the parameters (n, acc).

→ Both change. acc starts at 0 (from the wrapper); n is a parameter and already has its value.

```
public static long sumLoop(int n)
{
    long acc = 0;
}
```

Step 2 · Base case → loop condition

```
if (n == 0) return acc;
```

The recursion stops at `n == 0`. When does the loop keep going?

→ As long as that is **not** true: `n != 0`.

```
public static long sumLoop(int n)
{
    long acc = 0;
    while (n != 0)
    {
    }
}
```

Step 3 · Call → assignments

```
return sumAcc(n - 1, acc + n);
```

The next round works with `n - 1` and `acc + n`.

What if you swap the two lines?
→ Then acc adds the new, smaller n: `sumLoop(4)` adds $3 + 2 + 1 + 0$ and returns **6**. Compute every new value from the old values first.

```
public static long sumLoop(int n)
{
    long acc = 0;
    while (n != 0)
    {
        acc = acc + n;
        n = n - 1;
    }
}
```

Step 4 · Base return → after the loop

```java
public static long sumLoop(int n)
{
    long acc = 0;
    while (n != 0)
    {
        acc = acc + n;
        n = n - 1;
    }
    return acc;
}
```

The recursion returned acc at its base case. The loop ends exactly there, so it returns acc after the loop.

What you just did by hand is called **tail-call elimination**.

→ Some languages (Scheme) do it automatically. Java doesn't – even the tail-recursive `sumAcc(20000)` overflows. In Java, you write the loop yourself.

Code-along 4: run all three

→ **Switch to editor · Sum.java**

- Replace `main` in `Sum.java`: it calls `sumLoop`, `sumAcc` and `sum` for the `n` on the command line
- `try`/`catch` catches the `StackOverflowError`, so the loop's answer is still printed when a recursion crashes
- Run it with $n = 4$, 20000, 1000000000
- Which versions give an answer?

→ The loop, every time. Both recursions overflow at 20000 – the tail-recursive one, too.

Three traces side by side

→ **Switch to editor · demo · run SumTrace.java (watch, don't type)**

recursive

```
sum(4)
  sum(3)
    sum(2)
      sum(1)
        sum(0)
        returns 0
      returns 1
    returns 3
  returns 6
returns 10
```

accumulator

```
sumAcc(4, 0)
  sumAcc(3, 4)
    sumAcc(2, 7)
      sumAcc(1, 9)
        sumAcc(0, 10)
        returns 10
      returns 10
    returns 10
  returns 10
returns 10
```

loop

```
n = 4, acc = 0
n = 3, acc = 4
n = 2, acc = 7
n = 1, acc = 9
n = 0, acc = 10
returns 10
```

- → Recursive: the additions happen on the way **up** (1, 3, 6, 10)
- → Accumulator: done on the way **down**; every frame just passes 10 on
- → Loop: the same values as the accumulator calls – one call = one pass, without the frames. That's why the four steps are correct

The recipe

1. **Variables and start values:** parameters that change become loop variables; their first values come from the call that starts the recursion
2. **Base case → loop condition:** loop while the base case is *not* reached
3. **Call → assignments:** the call's arguments become the new values – compute from the old values first
4. **Base return → after the loop:** what the base case returned is returned after the loop

→ Work after the call? Do move 1 first: carry it along in an accumulator.

Now you: `BinarySearchLoop`

→ **Switch to editor · BinarySearchLoop.java, test client from class**

- Binary search is already tail-recursive: no move 1, only the four steps
- Write one method `search(String key, String[] a)` with a `while` loop
- Test with the client from class: `all 15 hits found`
- Closed book, no AI

Weekend: tutorial parts 5–7 (from a loop back to a recursion, which one to write, gcd).